

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Lectura #3 - Semana 4 – Capítulo 10
Aritmética

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 19/08/2024

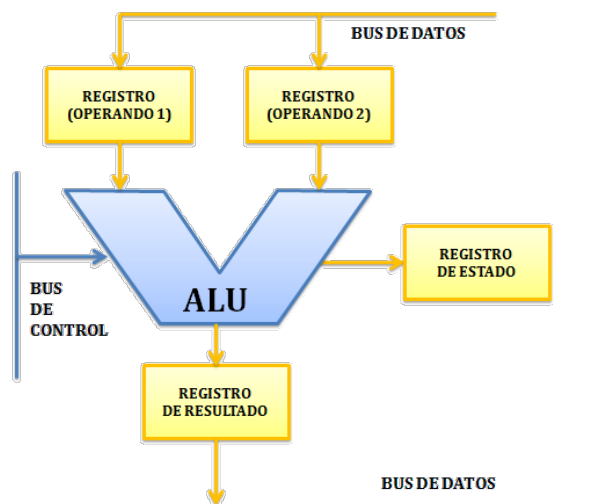
Bibliografía:

[1] W. Stallings, *Computer organization and architecture*, 9th ed. Pearson, 2012, pp. 319–359.

Abstract—The Arithmetic Logic Unit is a fundamental component of a computer system responsible for performing arithmetic and logical operations on data. Other system components, including the control unit, registers, memory, and I/O, primarily serve to deliver and manage data for processing by the ALU. Integer representation in computers relies on binary digits, and for positive integers, the representation is straightforward. However, for both positive and negative integers, various methods such as sign-magnitude and two's complement are used. Two's complement representation is preferred for its simplicity in arithmetic operations, allowing the same circuitry to handle both positive and negative numbers. Additionally, number range extension and fixed-point and floating-point representations cater to different precision and range requirements in computing. The IEEE 754-2008 standard outlines floating-point representations and arithmetic operations, ensuring accuracy and consistency across various platforms.

ALU

La Unidad Aritmética y Lógica es la parte del ordenador que realiza operaciones aritméticas y lógicas sobre los datos. Los demás componentes del sistema (unidad de control, registros, memoria, I/O) sirven principalmente para llevar datos a la ALU y procesarlos.



Representación de enteros

Para almacenar y procesar números en una computadora, solo se pueden usar dígitos binarios (0 y 1), sin símbolos especiales para el signo negativo o el punto decimal. Si se limitan a enteros no negativos, la representación es directa. Un número binario de n bits se interpreta como un entero sin signo calculando su valor como la suma de las potencias de 2 correspondientes a cada dígito.

Signo y magnitud

Es una forma de representar números enteros, tanto positivos como negativos, usando el bit más significativo como bit de signo. Si este bit es 0, el número es positivo; si es 1, es negativo. Esta representación tiene desventajas, como la necesidad de manejar los signos y magnitudes al realizar sumas y restas, y la existencia de dos representaciones diferentes para el número 0 ($+0 = 00000000$ y $-0 = 10000000$). Debido a estas limitaciones, la representación signo-magnitud se usa raramente, prefiriéndose en su lugar la representación en complemento a dos.

Complemento a dos

1. **Números Positivos:** Se representan de manera similar a la representación binaria estándar. Por ejemplo, en un sistema de 8 bits, el número 5 se representa como 00000101.
2. **Números Negativos:** Para representar un número negativo, se toma el valor absoluto del número, se convierte a binario, y luego se invierten todos los bits (se cambia 0 por 1 y viceversa). Finalmente, se suma 1 al resultado. Por ejemplo, para representar -5 en un sistema de 8 bits:

- El valor absoluto de 5 en binario es 00000101
 - Invertiendo los bits: 11111010
 - Sumando 1: 11111010
- $$\begin{array}{r} + \quad \quad \quad 1 \\ = \quad 11111011 \end{array}$$

En complemento a dos, el bit más significativo indica el signo: 0 para números positivos y 1 para números negativos. Esta representación facilita la realización de operaciones aritméticas como suma y resta, al permitir que el mismo circuito se use para números positivos y negativos.

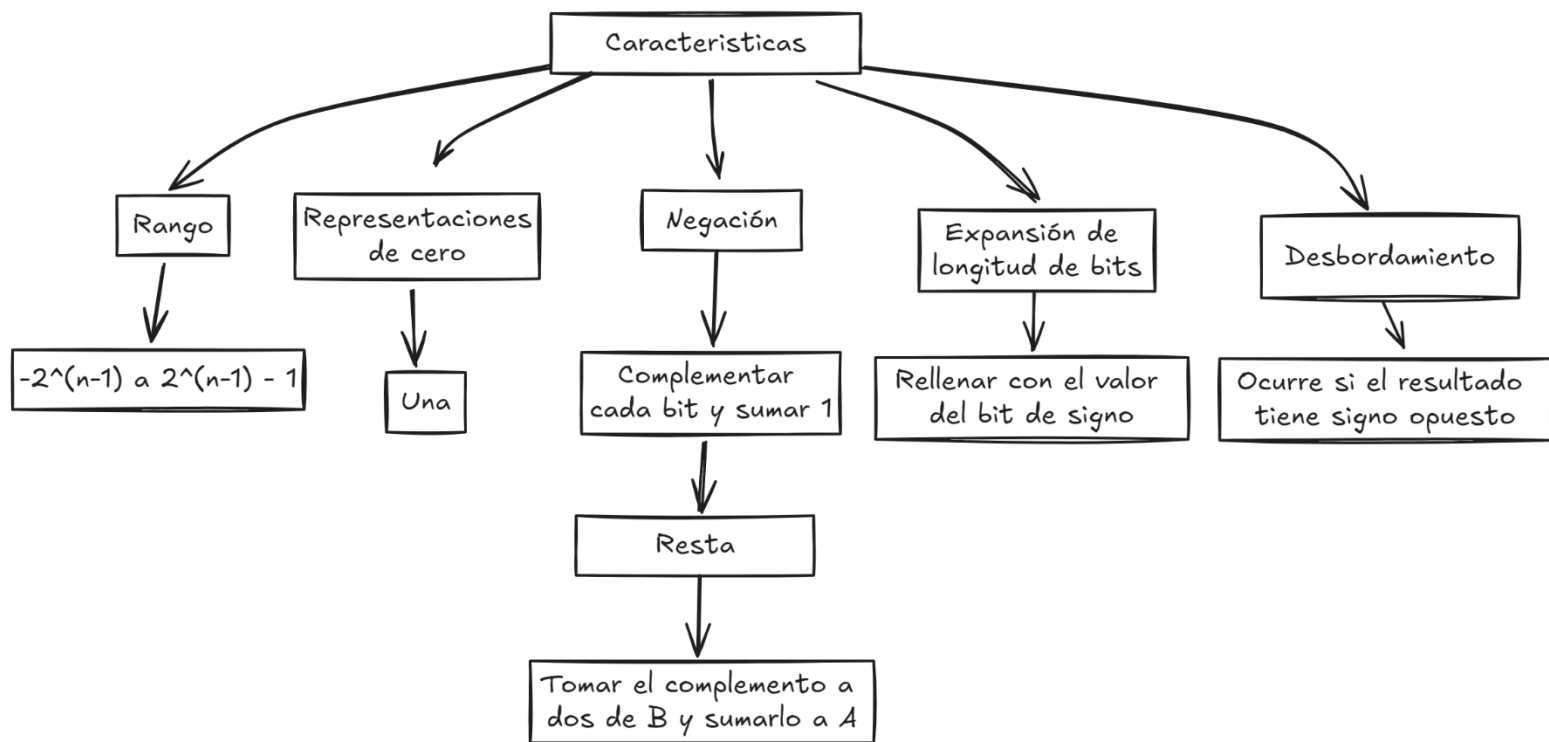


Tabla de representaciones

<u>Número decimal</u>	<u>Signo-Magnitud</u>	<u>2-complemento</u>
-8		1000
-7	1111	1001
-6	1110	1010
-5	1101	1011
-4	1100	1100
-3	1011	1101
-2	1010	1110
-1	1001	1111
0	1000	0000
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111

Extensión de rango

Es un proceso para aumentar la longitud en bits de un número entero, permitiendo representar un rango mayor de valores.

Para hacerlo con números en complemento a dos, se utiliza la extensión de signo. Aquí están los pasos:

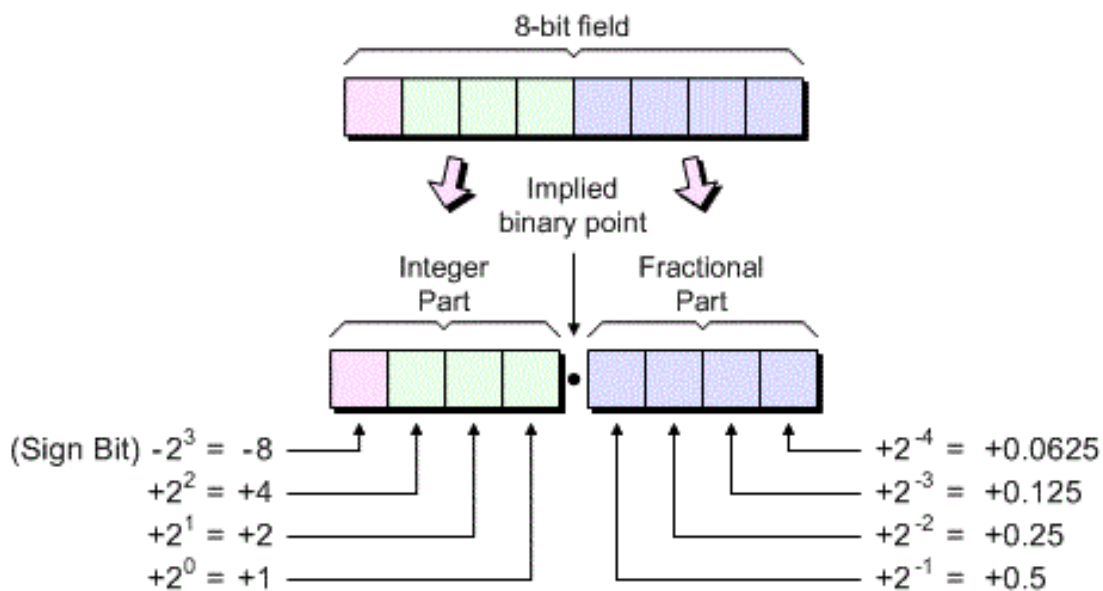
1. **Números Positivos:** Se extiende añadiendo ceros a la izquierda. Por ejemplo, para el número +18 en 8 bits (00010010), en 16 bits se representa como 0000000000010010.
2. **Números Negativos:** Se extiende duplicando el bit de signo. Por ejemplo, para -18 en 8 bits (11101110), en 16 bits se convierte en 1111111111101110.

Este método asegura que los números negativos se mantengan correctos en la nueva longitud de bits, manteniendo el mismo valor numérico al ampliar el rango.

Representación en punto fijo

En esta representación, el punto binario se encuentra siempre a la derecha del dígito más a la derecha, y no cambia de posición.

Para trabajar con fracciones binarias, se utiliza una representación en punto fijo escalando los números de modo que el punto binario se asuma en una posición específica dentro del número binario. Esto permite representar tanto enteros como fracciones utilizando la misma representación, simplemente ajustando el lugar donde se "coloca" el punto binario de manera implícita.



Aritmética en enteros con complemento a dos

Negación: En la representación en complemento a dos, la negación de un número entero se realiza de la siguiente manera:

1. **Complemento de bits:** Cambia cada bit del número (incluido el bit de signo) de 0 a 1 y viceversa.
2. **Suma de uno:** Añade 1 al resultado del complemento de bits.

Ejemplo: Para +18 en complemento a dos (en 8 bits):

- Representación en binario: 00010010
- Complemento de bits: 11101101
- Sumando 1: = 11101110, que representa -18.

Adición y Sustracción

Adición: La suma en complemento a dos se realiza como si se tratara de números enteros sin signo. Si hay un desbordamiento, el ALU debe detectarlo. La regla de desbordamiento es:

- Si ambos números son positivos o ambos negativos, el desbordamiento ocurre si el resultado tiene el signo opuesto.

Ejemplos:

$\begin{array}{r} 1001 = -7 \\ +0101 = 5 \\ \hline 1110 = -2 \\ (a) (-7) + (+5) \end{array}$	$\begin{array}{r} 1100 = -4 \\ +0100 = 4 \\ \hline 10000 = 0 \\ (b) (-4) + (+4) \end{array}$
$\begin{array}{r} 0011 = 3 \\ +0100 = 4 \\ \hline 0111 = 7 \\ (c) (+3) + (+4) \end{array}$	$\begin{array}{r} 1100 = -4 \\ +1111 = -1 \\ \hline 11011 = -5 \\ (d) (-4) + (-1) \end{array}$
$\begin{array}{r} 0101 = 5 \\ +0100 = 4 \\ \hline 1001 = \text{Overflow} \\ (e) (+5) + (+4) \end{array}$	$\begin{array}{r} 1001 = -7 \\ +1010 = -6 \\ \hline 10011 = \text{Overflow} \\ (f) (-7) + (-6) \end{array}$

Sustracción: Para restar un número de otro:

1. Calcula el complemento a dos del número a restar (sustriendo).
2. Suma este complemento al minuendo.

Ejemplos:

$\begin{array}{r} 0010 = 2 \\ +1001 = -7 \\ \hline 1011 = -5 \end{array}$ (a) M = 2 = 0010 S = 7 = 0111 -S = 1001	$\begin{array}{r} 0101 = 5 \\ +1110 = -2 \\ \hline 10011 = 3 \end{array}$ (b) M = 5 = 0101 S = 2 = 0010 -S = 1110
$\begin{array}{r} 1011 = -5 \\ +1110 = -2 \\ \hline 11001 = -7 \end{array}$ (c) M = -5 = 1011 S = 2 = 0010 -S = 1110	$\begin{array}{r} 0101 = 5 \\ +0010 = 2 \\ \hline 0111 = 7 \end{array}$ (d) M = 5 = 0101 S = -2 = 1110 -S = 0010
$\begin{array}{r} 0111 = 7 \\ +0111 = 7 \\ \hline 1110 = \text{Overflow} \end{array}$ (e) M = 7 = 0111 S = -7 = 1001 -S = 0111	$\begin{array}{r} 1010 = -6 \\ +1100 = -4 \\ \hline 10110 = \text{Overflow} \end{array}$ (f) M = -6 = 1010 S = 4 = 0100 -S = 1100

Multiplicación

La multiplicación de números binarios sin signo es similar a la multiplicación de números decimales, pero en lugar de multiplicar dígitos del 0 al 9, se usa solo los dígitos 0 y 1.

Pasos:

1. Genera los productos parciales:

- Se multiplica cada bit del multiplicador (el segundo número) por el multiplicando (el primer número). Debido a que se está trabajando con números binarios, cada producto parcial será igual al multiplicando si el bit del multiplicador es 1, o será 0 si el bit del multiplicador es 0.
- Se coloca cada producto parcial en la posición correcta, desplazando hacia la izquierda para cada siguiente bit del multiplicador.

2. Suma los productos parciales:

- Se suman todos los productos parciales para obtener el resultado final.

Ejemplo:

$$1011_2 \times 1101_2 =$$

1011	Multiplicando (11 en decimal)
x 1101	Multiplicador (13 en decimal)

1011	Primer producto parcial ($1011 * 1$)
0000	Segundo producto parcial ($1011 * 0$, desplazado 1 bit a la izquierda)
1011	Tercer producto parcial ($1011 * 1$, desplazado 2 bits a la izquierda)
1011	Cuarto producto parcial ($1011 * 1$, desplazado 3 bits a la izquierda)

= 10001111	Suma de los productos parciales (143 en decimal)

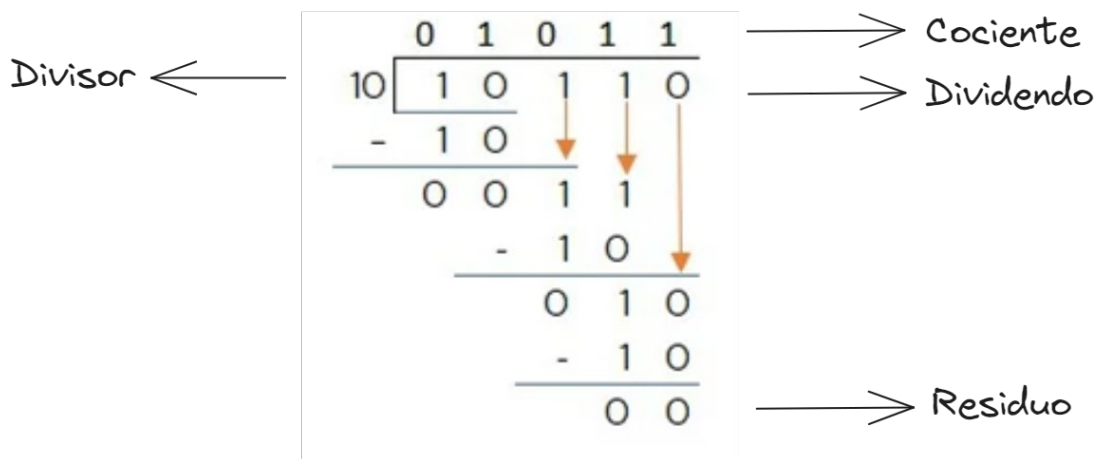
El proceso implica multiplicar cada bit del multiplicador por el multiplicando y luego sumar estos productos parciales desplazados. El producto resultante será de hasta $2n$ bits, donde n es el número de bits en el multiplicando y el multiplicador.

En el caso de números con signo, se utiliza la representación en complemento a 2:

- Si ambos números son positivos, se realiza la multiplicación directamente y se añade un bit de signo 0 al resultado (indicando que el resultado es positivo).
- Si ambos números son negativos, se convierten a su complemento a 2 antes de multiplicarlos, y luego se añade un bit de signo 0 al resultado (indicando que el resultado es positivo).
- Si uno de los números es negativo, se convierte ese número a su complemento a 2, se multiplica por el otro número, y al resultado se le aplica el complemento a 2, añadiendo un bit de signo 1 (indicando que el resultado es negativo).

División

La división se realiza también como en sistema decimal:



En el caso de números con signos se utiliza la misma representación que en la multiplicación.

Representación de Números Reales

Punto Fijo: En la representación de punto fijo, la posición de la coma decimal (o punto binario) está fija. Esto significa que el número se divide en una parte entera y una parte fraccionaria que se mantienen constantes en longitud.

- **Formato:**
 - **Qm,n:** Notación utilizada para describir el formato de punto fijo.
 - **m:** Número de bits para la parte entera.
 - **n:** Número de bits para la parte decimal.
 - **Total de bits:** $m + n + 1$ (incluyendo el bit para el signo).
- **Ejemplo:**
 - **Q16,16:** 16 bits para la parte entera y 16 bits para la parte decimal.
 - **Q32:** Si se usa esta notación sin especificar m y n, se podría asumir que se usa el total de 32 bits para representar el número, de forma similar a **Q16,16** si se divide en partes enteras y decimales en proporciones específicas.
- **Ventajas:**
 - **Simplicidad:** Operaciones aritméticas como sumas, restas, multiplicaciones y divisiones son directas y no requieren manejo de exponente.
- **Limitaciones:**
 - **Rango fijo:** El rango de números que se pueden representar es limitado por el número de bits asignados a la parte entera y fraccionaria. No puede manejar números extremadamente grandes o fracciones muy pequeñas.

Punto Flotante: En la representación de punto flotante, la coma decimal puede “flotar”. Esto significa que los números se representan en la forma de una mantisa y un exponente. La mantisa representa el valor significativo del número, mientras que el exponente ajusta la posición del punto decimal.

- **Formato:**
 - **$N = M \times b^E$**
 - **M:** Mantisa.

- **b:** Base del sistema numérico (generalmente 2 en binario).
- **E:** Exponente.
- **Ejemplo:**
 - **2547.35** en notación científica decimal es 2.54735×10^3
 - **0.0035** es 3.5×10^{-3}
 - **111.0110** en binario es 1.1100110×2^6
 - **0.001101** en binario es 1.101×2^{-3}
- **Normalización:**
 - La normalización asegura que la mantisa esté en su forma estándar, típicamente con un bit más significativo en una posición fija (como 1 en la base 2), lo que simplifica el almacenamiento y las operaciones. Esto evita la redundancia en la representación.
- **Ventajas:**
 - **Rango Amplio:** Puede representar números extremadamente grandes o pequeños.
 - **Flexibilidad:** Adapta la precisión y el rango al contexto del cálculo.
- **Limitaciones:**
 - **Precisión:** Los números no están distribuidos uniformemente, lo que puede llevar a errores de redondeo en cálculos.
 - **Complejidad:** Las operaciones aritméticas son más complejas debido a la necesidad de manejar exponentes y ajustar la mantisa.

El punto fijo es simple y adecuado para cálculos con un rango fijo, mientras que el punto flotante ofrece flexibilidad y un amplio rango de representación a expensas de una mayor complejidad.

IEEE Standard 754-2008: Representación Binaria de Punto Flotante

Tipos de Formatos Definidos

1. **Formato Aritmético:** Formatos que soportan todas las operaciones aritméticas obligatorias definidas por el estándar.
 - **Uso:** Representar operandos o resultados para las operaciones descritas.
2. **Formato Básico:** Incluye cinco representaciones de punto flotante (tres binarias y dos decimales) especificadas por el estándar.

- **Tipos:**
 - **Binario:** 32 bits, 64 bits y 128 bits, con exponentes de 8, 11 y 15 bits respectivamente.
 - **Decimal:** 64 bits y 128 bits.
 - **Características:** Cada formato puede ser usado tanto para aritmética como para intercambio de datos entre plataformas.
3. **Formato de Intercambio:**
- **Definición:** Codificación binaria fija que permite el intercambio de datos entre diferentes plataformas y puede usarse para almacenamiento.
 - **Ejemplos:** Formatos de 32, 64 y 128 bits.
4. **Formatos Extendidos:** Proporcionan bits adicionales en el exponente y significando para mayor precisión y rango.
- **Uso:** Cálculos intermedios para reducir errores por redondeo y evitar desbordamiento.
5. **Formato de Precisión Extendida:** Formato cuyo nivel de precisión y rango es controlado por el usuario.
- **Uso:** Similar a los formatos extendidos, pero sin restricciones estrictas sobre el formato o longitud.

Valores Especiales

- **Normalización:**
 - **Valores de Exponente:**
 - **0 a 254** (32 bits), **0 a 2046** (64 bits), **0 a 16382** (128 bits): Números flotantes normales.
 - **Exponente sesgado:** El rango es -126 a +127 para el formato de 32 bits, etc.
 - **Mantisa efectiva:** 24 bits, 53 bits, o 113 bits dependiendo del formato.
- **Cero:**
 - **Exponente 0 y fracción 0:** Representa cero positivo o negativo según el bit de signo.

- **Infinito:**
 - **Exponente todos unos y fracción 0:** Representa infinito positivo o negativo según el bit de signo.
- **Números Subnormales:**
 - **Exponente 0 y fracción no cero:** Representa números subnormales donde el bit a la izquierda del punto binario es cero.
- **NaN (Not a Number):**
 - **Exponente todos unos y fracción no cero:** Utilizado para indicar excepciones o condiciones de error.

Detalles de representación

- **Signo:** 1 bit para indicar el signo (positivo o negativo).
- **Exponente:** Un número fijo de bits para representar el exponente.
- **Mantisa:** Un número fijo de bits para la parte fraccionaria significativa.

Resumen de Formatos

- **Formato de 32 bits:** 1 bit de signo, 8 bits de exponente, 23 bits de mantisa.
- **Formato de 64 bits:** 1 bit de signo, 11 bits de exponente, 52 bits de mantisa.
- **Formato de 128 bits:** 1 bit de signo, 15 bits de exponente, 112 bits de mantisa.

Aritmética de Punto Flotante

La aritmética de punto flotante involucra operaciones básicas que requieren manejo especial para asegurar precisión y correcta representación. Aquí se resume el proceso para realizar operaciones aritméticas con números en formato de punto flotante según el estándar IEEE 754-2008.

1. Suma y Resta

- **Fase 1: Verificación de ceros:** Se verifica si alguno de los operandos es cero. Si uno de ellos es cero, el resultado es el otro operando.
- **Fase 2: Alineación de significandos:** Para operar, los significandos deben tener el mismo exponente. Esto se logra ajustando el punto decimal (radix) de uno de los operandos hasta que ambos exponentes coincidan.
 - **Ejemplo:** Para sumar 123×10^0 y 456×10^{-2} se ajusta el segundo número a 4.56×10^0 para que los exponentes coincidan.

- **Fase 3: Suma o resta de significandos:** Una vez alineados, los significandos se suman o restan según corresponda. Se maneja el desbordamiento del significando si ocurre.
- **Fase 4: Normalización:** Se normaliza el resultado ajustando el significando para que el dígito más significativo sea no cero. Si la normalización resulta en un desbordamiento de exponente, este se reporta y el resultado se ajusta en consecuencia.

Multiplicación

Fórmula: $X \times Y = (X_S \times Y_S) \times B^{(X_E + Y_E)}$

La multiplicación de números en punto flotante es relativamente simple en comparación con la suma y la resta. El proceso se descompone en los siguientes pasos:

1. **Verificación de cero:** Si cualquiera de los operandos es cero, el resultado es cero.
2. **Suma de exponentes:** Se suman los exponentes de los dos operandos. Si los exponentes están en forma sesgada (biased), el sesgo debe restarse del resultado de la suma de exponentes para obtener el exponente del producto. Si el exponente resultante está fuera del rango representable, se reporta un desbordamiento o subdesbordamiento de exponente.
3. **Multiplicación de significandos:** Se multiplican los significandos de los operandos, teniendo en cuenta sus signos. La multiplicación se realiza como con enteros, pero el producto será el doble de largo que los operandos, y los bits adicionales se perderán durante el redondeo.
4. **Normalización y Redondeo:** El resultado se normaliza y se redondea para ajustar el significando a la longitud requerida. La normalización puede causar un subdesbordamiento de exponente si se requieren más dígitos significativos.

División

Fórmula: $X \div Y = \left(\frac{X_S}{Y_S} \right) \times B^{(X_E - Y_E)}$

La división también sigue un proceso claro:

1. **Verificación de Cero**
 - Si el divisor es cero, se emite un informe de error o se establece el resultado como infinito, según la implementación. Si el dividendo es cero, el resultado es cero.
2. **Sustracción de Exponentes**

- Se resta el exponente del divisor del exponente del dividendo. Si el exponente resultante está fuera del rango representable, se reporta un desbordamiento o subdesbordamiento.

3. División de Significandos

- Se dividen los significandos de los operandos, seguido de la normalización y redondeo.

Condiciones que pueden surgir:

- **Desbordamiento del Exponente:** Un exponente positivo excede el máximo valor permitido. En algunos sistemas, esto se representa como $+\infty$.
- **Subdesbordamiento del Exponente:** Un exponente negativo es menor que el mínimo valor permitido, lo que puede llevar a que el número se reporte como cero.
- **Subdesbordamiento del Significante:** Durante la alineación, pueden perderse dígitos del significando. Se requiere redondeo para ajustar el resultado.
- **Desbordamiento del significando:** La adición de dos significandos del mismo signo puede causar un desbordamiento. Se debe ajustar el significando y posiblemente el exponente.

Consideraciones de Precisión

1. **Bits de guardia:** Los bits de guardia son bits adicionales en el registro que se utilizan para evitar la pérdida de precisión en los cálculos. Se utilizan para almacenar dígitos adicionales al final del significando durante las operaciones para minimizar la pérdida de precisión. Esto es importante, especialmente cuando los números son muy cercanos en valor.
 - **Ejemplo:** Si se resta y de x y y pierde un bit de significancia, el uso de bits de guardia puede evitar esta pérdida y mejorar la precisión del resultado.
2. **Redondeo:** El redondeo es el proceso de ajustar el resultado a un número representable cuando el resultado de una operación tiene más bits de los que se pueden almacenar. El estándar IEEE define cuatro métodos de redondeo:
 - **Redondeo al más cercano:** Redondea al número representable más cercano.
 - **Redondeo hacia $+\infty$:** Redondea hacia infinito positivo.
 - **Redondeo hacia $-\infty$:** Redondea hacia infinito negativo.
 - **Redondeo hacia 0:** Redondea hacia cero.
 - **Redondeo al más cercano:** Si los bits extra están más allá de la mitad de la última posición representable, se redondea hacia arriba. Si están

menos de la mitad, se redondea hacia abajo. En caso de estar exactamente en el medio, el redondeo se realiza hacia el número par más cercano (también conocido como redondeo al par más cercano).

- **Redondeo hacia infinito:** Se utiliza para la aritmética de intervalos, donde se calculan dos valores para cada resultado, proporcionando un intervalo que contiene el verdadero resultado. Esto ayuda a monitorear y controlar los errores en cálculos de punto flotante.
- **Redondeo hacia cero:** Simplemente se truncan los bits adicionales, lo que introduce un sesgo consistente hacia cero.

Ejemplos:

Suma:

$$X = 0.3 \times 10^2$$

$$Y = 0.2 \times 10^3$$

$$X + Y = (0.3 \times 10^{-1} + 0.2) \times 10^3 = 0.23 \times 10^3 = 230$$

Resta:

$$X - Y = (0.3 \times 10^{-1} - 0.2) \times 10^3 = (-0.17) \times 10^3 = -170$$

Multiplicación:

$$X \times Y = (0.3 \times 0.2) \times 10^{2+3} = 0.06 \times 10^5 = 6000$$

División:

$$X \div Y = \frac{0.3}{0.2} \times 10^{2-(-3)} = 1.5 \times 10^{-1} = 0.15$$

IEEE 754: Estándar de aritmética de punto flotante binario

El estándar IEEE 754 define prácticas y procedimientos para garantizar resultados uniformes y predecibles en la aritmética de punto flotante, independientemente de la plataforma de hardware. Además del redondeo, el estándar aborda tres temas adicionales importantes: infinito, NaNs (Not a Number) y números subnormales.

Infinito: En la aritmética de punto flotante, los valores infinitos representan el caso límite de los números reales y se interpretan de la siguiente manera:

- $-\infty$ (todo número finito) $\leq +\infty$

Las operaciones aritméticas que involucran infinito siguen reglas simples y lógicas:

- $5 + (+\infty) = +\infty$
- $(+\infty) + (+\infty) = +\infty$
- $5 - (+\infty) = -\infty$
- $(+\infty) + (-\infty) = \text{indeterminado}$
- $(+\infty) - (-\infty) = +\infty$ $(+\infty) - (-\infty) = +\infty$
- $5 \times (+\infty) = +\infty$

- $(+\infty) \times (-\infty) = -\infty$
- Los resultados pueden variar en casos especiales, como operaciones entre infinitos y cero.

NaNs (Not a Number): Un NaN es una entidad simbólica codificada en formato de punto flotante y existen dos tipos principales:

- **NaNs Significantes:** Señalan una excepción de operación inválida cuando aparecen como operandos. Se utilizan para representar valores no inicializados o en contextos donde se requiere una señalización explícita de errores.
- **NaNs Silenciosos:** Propagan a través de casi todas las operaciones aritméticas sin señalar una excepción. Son útiles para evitar la interrupción del cálculo cuando se encuentran durante la ejecución.

Ambos tipos de NaNs tienen el mismo formato general: un exponente de todos unos y una fracción no cero. La diferencia en el patrón de bits de la fracción permite distinguir entre NaNs significantes y silenciosos.

Números Subnormales: Los números subnormales se introducen para manejar casos de subdesbordamiento del exponente. Cuando el exponente de un resultado se vuelve demasiado pequeño (un exponente negativo con una magnitud excesiva), el resultado se subnormaliza al desplazar la fracción hacia la derecha y aumentar el exponente con cada desplazamiento hasta que el exponente esté dentro del rango representable.

Efectos de los Números Subnormales:

- Los números representables se agrupan en intervalos de la forma $[2^n, 2^{n+1}]$. Dentro de cada intervalo, la porción del exponente permanece constante mientras que la fracción varía, produciendo una separación uniforme de números representables dentro del intervalo.
- A medida que nos acercamos a cero, cada intervalo sucesivo es la mitad del ancho del intervalo anterior, pero contiene el mismo número de números representables. La densidad de números representables aumenta a medida que nos acercamos a cero.
- En el formato IEEE 754 de 32 bits, hay 223 números representables en cada intervalo. El número representable positivo más pequeño es 2^{-126} . Con los números subnormales, se añaden 223 - 1 números uniformemente entre 0 y 2^{-126} .

Subnormalidad Gradual: La inclusión de números subnormales evita un hueco significativo entre el número positivo más pequeño representable y cero, reduciendo el impacto del subdesbordamiento del exponente a un nivel comparable con el redondeo entre los números normales.